



RFC: Combat Death Tracking

Via Local Poller

RFC ID: RFC-DOO-0002

Status: Draft

Date: 06 July 2026

Authors: DarkDante (@yacketrj)

Target: dune-awakening-selfhost-docker core

Abstract

The dune database does not store cumulative combat death data. The `event_log` table (31 partitions) and `game_events` table record building/interaction events but contain zero combat-related rows. Funcom's backend (FLS) aggregates per-server death/kill metrics server-side but exposes no public query API.

This RFC proposes a lightweight local poller that snapshots `player_state.life_state` transitions to populate a `player_death_log` table, enabling the addon's Combat tab via the `ops.combat.deaths` bridge action.

Motivation

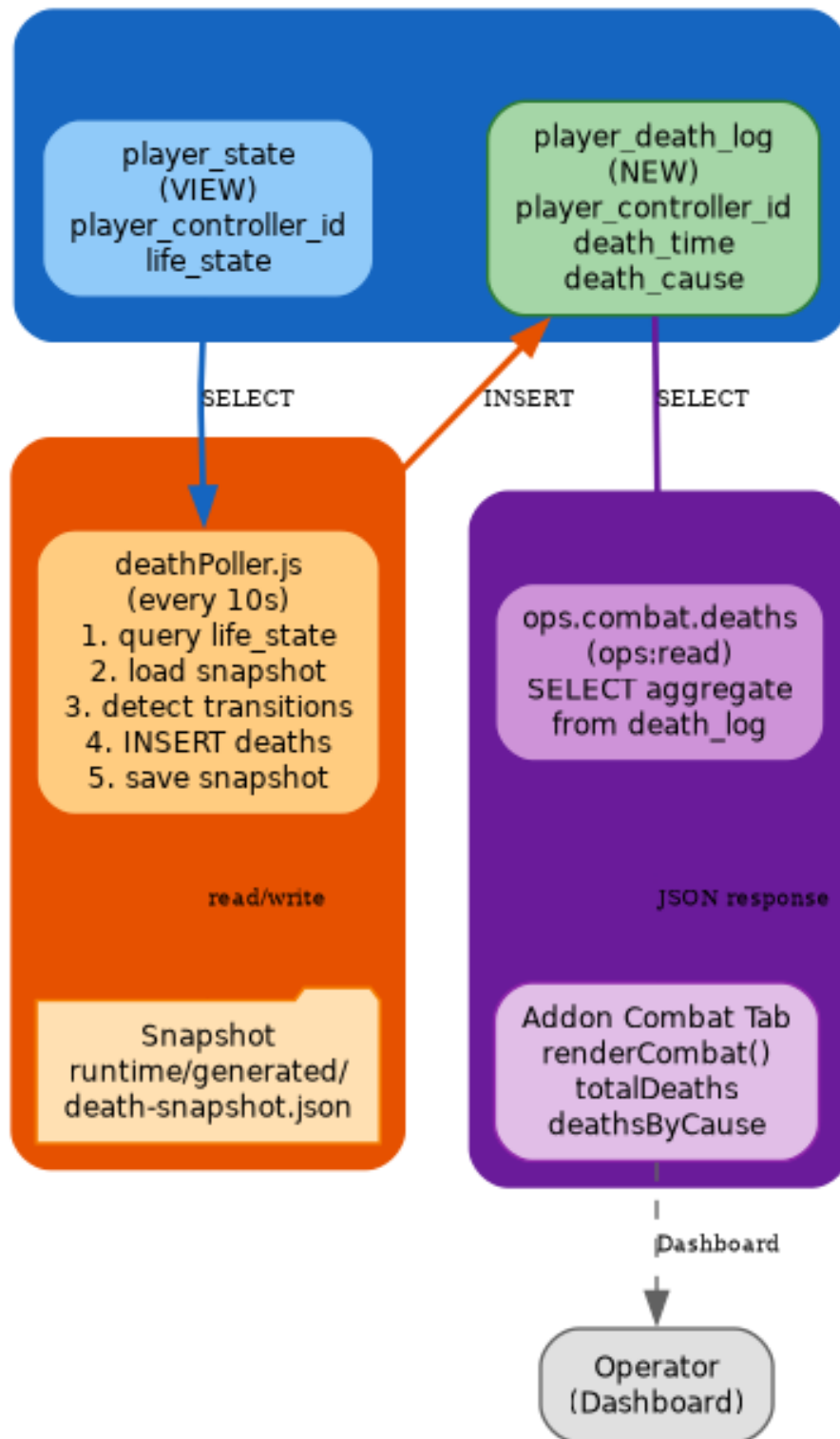
Current State

Source	Status	Reason
<code>event_log</code> (31 partitions)	0 rows	Only logs economy transactions (solaris category)
<code>game_events</code>	3 rows	Building/totem events only — no combat
<code>actor_audit</code>	27 rows	Building actor lifecycle — no deaths
<code>player_state.life_state</code>	Current-state	Shows Alive/Dead — no history
Funcom FLS	Backend-only	Director pushes data, no query API

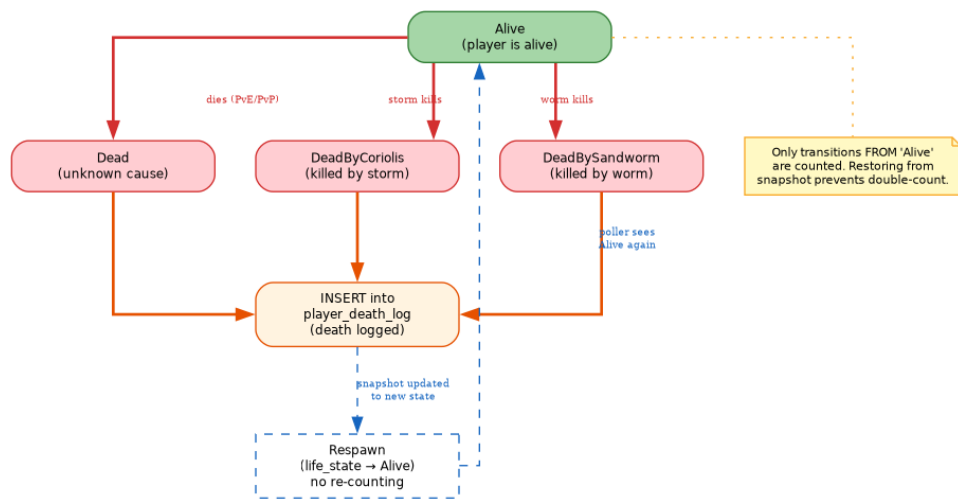
Analysis

The game engine tracks death causes natively — the `playerlifestate` enum includes `Dead`, `DeadByCoriolis`, and `DeadBySandworm`. However, none of these are persisted as cumulative counters locally. The data is sent to Funcom's backend where they aggregate it for their public metrics visual. There is no API to retrieve this data back.

Proposed Architecture



Data flow: `player_state` → death poller → `player_death_log` → bridge → addon Combat tab



State machine: Alive → Dead transition detection with idempotency

Schema

```

CREATE TABLE IF NOT EXISTS dune.player_death_log (
  id bigint GENERATED ALWAYS AS IDENTITY,
  player_controller_id bigint NOT NULL,
  death_time timestamptz NOT NULL DEFAULT now(),
  death_cause text NOT NULL,
  PRIMARY KEY (id)
);

```

The death_cause column stores the raw life_state::text value: Dead, DeadByCoriolis, DeadBySandworm. No PII is stored — only player_controller_id (internal identifier).

Component Specifications

deathPoller.js

New module at console/api/src/deathPoller.js, following the exact pattern of memoryBalancer.js:

- In-process: setInterval with .unref() — no separate process or container
- State: runtime/generated/death-snapshot.json stores last known life_state per player
- Lock boolean prevents overlapping polls
- Error suppression for transient DB errors (ECONNREFUSED, etc.)
- Interval: 10 seconds (configurable via ADMIN_DEATH_POLL_INTERVAL_MS)

addonOpsCombatDeaths() in duneDb.js

Queries player_death_log for aggregate counts. Returns:

- totalDeaths, pvpDeaths (0), pveDeaths
- deathsByCause: [{ cause: 'Sandworm', count: N }, ...]
- deathsByMap: [] — not tracked
- topHostileNpcs: [] — NPC kill data is backend-only
- kdRatio: null — no kill count available

Bridge Route

ops.combat.deaths follows the exact same pattern as ops.activity.summary: permission check (ops:read), query, audit log, 200 JSON response.

Security Review

Data Exposure

Field	Classification	Rationale
player_controller_id	Internal only	Used for snapshot diffing only — not in responses
death_time	Aggregate only	Bridge returns counts grouped by cause
death_cause	Safe aggregate	Enum string — no PII, no coordinates
Bridge response	Aggregate counts	Zero individual player data

Attack Surface

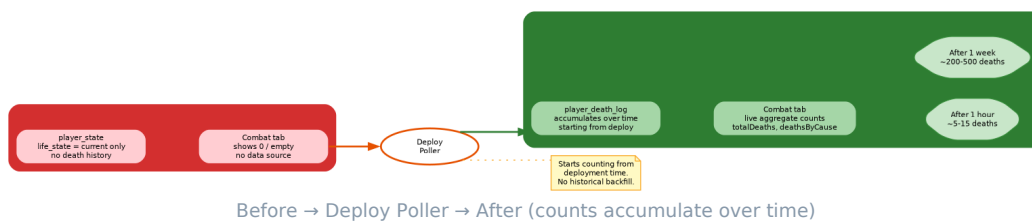
Concern	Mitigation
SQL injection	Parameterized db.query(text, values) — no string interpolation
Table creation	dune user already has ALL PRIVILEGES on dune schema
Write access	Poller is sole INSERT source. Bridge is SELECT-only
Rate limiting	Bridge inherits bridgeRateLimiter (60/min, 300/min global)
Audit	Every bridge call audited to web-admin-audit.jsonl
Idempotency	Snapshot-based diffing prevents double-counting on restart
Snapshot file	Written with mode 0o600

Non-Negotiable Rules (per release-standard.md): raw player rows, player IDs, account IDs, character names, coordinates, SQL text, tokens, passwords, secrets — none are exposed through this bridge action.

Limitations

Limitation	Impact	Mitigation
No historical backfill	Deaths before deployment not captured	Counts start from deploy time
No PvP vs PvE	pvpDeaths always 0	life_state doesn't distinguish cause
No NPC kill tracking	topHostileNpcs always empty	NPC kills are Funcom backend-only
No map tracking	deathsByMap always empty	death location not in player_state
Single process	Poller stops if Console restarts	Snapshot prevents double-counting

Deployment Timeline



Phases

- Phase 1: Deploy poller + bridge action to e2e. Verify combustion tab populates.
- Phase 2: Submit upstream PR with RFC as supporting evidence.
- Phase 3: Open upstream issue requesting Funcom FLS metrics API for historical data.

Open Issues

- Funcom FLS metrics endpoint — request per-server query API
- PvP classification — explore cheater_tracking, encounters_static tables
- NPC kill tracking — enhance actor_audit for NPC death events
- Polling interval tuning — default 10s may miss rapid death cycles
- Retention policy — player_death_log grows unbounded

This RFC is open for review via PR or issue on [yacketrj/dune-awakening-selfhost-docker](#). Last updated: 06 July 2026.